Stocks Buy and Sell Problem: Variant 1

Maximizing Profit with a Single Transaction

```
^ Price
|    /\
|   /  \
|  /____\
+-----------> Day
```

Introduction

- This video discusses the *Stocks Buy and Sell* problem, focusing on *Variant 1*: buying and selling a single stock.
- The problem involves an array of stock prices for different days.
- **Key Assumption (for this problem):** You have *foreknowledge of future prices*. This is not a real-world scenario, but a problem simplification.
- **Goal:** Determine the optimal day to buy and optimal day to sell *one stock* to maximize profit.
- You must return the *maximum profit*.

Problem Description

- **Input:** An array representing stock prices on consecutive days.

  ```
  [Price_Day1, Price_Day2, ..., Price_DayN]
  ```

- **Output:** The maximum possible profit.
- **Constraint:**
  - You must *buy before you sell*. You cannot sell a stock before purchasing it.
  - Example: If prices are [..., 9 (sell), ..., 1 (buy), ...], this is invalid. The buy must occur on or before the sell day.

Approaches to Solution

1. Brute Force Solution 🐻

```
  O(N^2)
+-----------+
| Outer Loop| (Buy Day)
|   +-------+
|   | Inner | (Sell Day)
|   | Loop  |
|   +-------+
```

```
+-----------+
```

- **Idea:** Iterate through every possible buy day and, for each buy day, iterate through every subsequent day to find a potential sell day.
- **Process:**
    1. Pick a day `i` to *buy* the stock.
    2. For all days `j` *after* day `i`, pick day `j` to *sell* the stock.
    3. Calculate `profit = price[j] - price[i]`.
    4. Keep track of the `maximum profit` found across all pairs.
- **Time Complexity:** *O(n^2)* (due to two nested loops).
- **Space Complexity:** *O(1)* (constant extra space).

2. Optimized using Auxiliary Array (Space-Time Trade-off) 💡

```
Aux Array: [Max_Future_Price]
           ^
           | Pre-processed
```

- **Idea:** Pre-process the array to store the *maximum future price* for each day. This allows for a single pass to calculate profit.
- **Steps:**
    1. Create an *auxiliary array* (e.g., `max_prices_ahead`) of the same size as the input prices array.
    2. Populate `max_prices_ahead` from *right to left*:
        - `max_prices_ahead[last_day] = prices[last_day]`
        - For `i` from `last_day - 1` down to `0`: `max_prices_ahead[i] = max(prices[i], max_prices_ahead[i+1])`
        - This means `max_prices_ahead[i]` stores the maximum price from day `i` *onwards*.
    3. Iterate through the original `prices` array from *left to right* (from day `0` to `n-1`).
    4. For each day `i`:
        - Calculate `potential_profit = max_prices_ahead[i] - prices[i]`. (Here, `max_prices_ahead[i]` represents the best selling price if you buy on day `i`).
        - Update the overall `maximum profit` found so far.
- **Example Dry Run** (with prices: `[3, 1, 4, 8, 7, 2, 5]`)
    - `max_prices_ahead` array creation (right to left):
        - Day 6 (Price 5): `max_prices_ahead[6] = 5`
        - Day 5 (Price 2): `max_prices_ahead[5] = max(2, 5) = 5`
        - Day 4 (Price 7): `max_prices_ahead[4] = max(7, 5) = 7`
        - Day 3 (Price 8): `max_prices_ahead[3] = max(8, 7) = 8`
        - Day 2 (Price 4): `max_prices_ahead[2] = max(4, 8) = 8`
        - Day 1 (Price 1): `max_prices_ahead[1] = max(1, 8) = 8`
        - Day 0 (Price 3): `max_prices_ahead[0] = max(3, 8) = 8`

      So, `max_prices_ahead = [8, 8, 8, 8, 7, 5, 5]`
    - Calculating `maxProfit` (left to right):
        - Day 0 (Price 3): `profit = max_prices_ahead[0] - 3 = 8 - 3 = 5`. `maxProfit = 5`.

        - Day 1 (Price 1): `profit = max_prices_ahead[1] - 1 = 8 - 1 = 7`. `maxProfit = max(5, 7) = 7`.
        - Day 2 (Price 4): `profit = max_prices_ahead[2] - 4 = 8 - 4 = 4`. `maxProfit = max(7, 4) = 7`.
        - Day 3 (Price 8): `profit = max_prices_ahead[3] - 8 = 8 - 8 = 0`. `maxProfit = max(7, 0) = 7`.
        - Day 4 (Price 7): `profit = max_prices_ahead[4] - 7 = 7 - 7 = 0`. `maxProfit = max(7, 0) = 7`.
        - Day 5 (Price 2): `profit = max_prices_ahead[5] - 2 = 5 - 2 = 3`. `maxProfit = max(7, 3) = 7`.
        - Day 6 (Price 5): `profit = max_prices_ahead[6] - 5 = 5 - 5 = 0`. `maxProfit = max(7, 0) = 7`.
    - **Final `maxProfit = 7`**.
- **Time Complexity:** *O(n)* (one pass for auxiliary array, one pass for profit calculation).
- **Space Complexity:** *O(n)* (for the auxiliary array).

3. Optimal Solution (Linear Time, Constant Space) ✨

```
+------------------+
| O(N) Time, O(1) Space |
+------------------+
|  minSoFar        |
|  maxProfit       |
+------------------+
```

- **Goal:** Solve the problem in *O(n) time complexity* and *O(1) space complexity*. This is the ideal solution for interviews.
- **Idea:** When selling on a particular day, the maximum profit is achieved by buying at the *lowest price encountered so far* before or on that day.
- **Variables:**
    - `minSoFar`: Stores the minimum stock price encountered from the beginning of the array up to the *current day*.
    - `maxProfit`: Stores the maximum profit calculated so far.
- **Steps:**
    1. Initialize `maxProfit = 0`.
    2. Initialize `minSoFar = prices[0]` (or a very large number like `infinity`).
    3. Iterate through the `prices` array from the *beginning* (or second element if `minSoFar` initialized with `prices[0]`).
    4. For each `current_price` on day `i`:
        - Update `minSoFar = min(minSoFar, current_price)`. (Ensures `minSoFar` always holds the lowest price up to the current day.)
        - Calculate `current_profit = current_price - minSoFar`. (This represents the profit if we sell today and bought at the lowest point seen so far.)
        - Update `maxProfit = max(maxProfit, current_profit)`. (Keep track of the highest profit found.)
    5. After iterating through all prices, `maxProfit` will hold the maximum possible profit.
- **Example Dry Run** (with prices: `[5, 2, 6, 1, 4]`)
    - Initialize: `maxProfit = 0`, `minSoFar = 5` (first price)
    - Day 0 (Price 5):

        - `minSoFar = min(5, 5) = 5`
        - `current_profit = 5 - 5 = 0`
        - `maxProfit = max(0, 0) = 0`
    - Day 1 (Price 2):
        - `minSoFar = min(5, 2) = 2`
        - `current_profit = 2 - 2 = 0`
        - `maxProfit = max(0, 0) = 0`
    - Day 2 (Price 6):
        - `minSoFar = min(2, 6) = 2`
        - `current_profit = 6 - 2 = 4`
        - `maxProfit = max(0, 4) = 4`
    - Day 3 (Price 1):
        - `minSoFar = min(2, 1) = 1`
        - `current_profit = 1 - 1 = 0`
        - `maxProfit = max(4, 0) = 4`
    - Day 4 (Price 4):
        - `minSoFar = min(1, 4) = 1`
        - `current_profit = 4 - 1 = 3`
        - `maxProfit = max(4, 3) = 4`
    - **Final `maxProfit = 4`**. (This profit is achieved by buying at price 2 and selling at price 6).
- **Time Complexity:** *O(n)* (single pass through the array).
- **Space Complexity:** *O(1)* (only two variables are used).

Conclusion

- The *optimal solution* using `minSoFar` and `maxProfit` variables is the most efficient and preferred method.
- This covers the first variant of the Stocks Buy and Sell problem. Other variants (e.g., buying/selling multiple stocks) will be discussed separately.